Type Conversions

11 April 2026 11:55

Most of the time, operators and functions automatically convert the values given to them to the right type.
For example, alert automatically converts any value to a string to show it. Mathematical operations convert values to numbers.

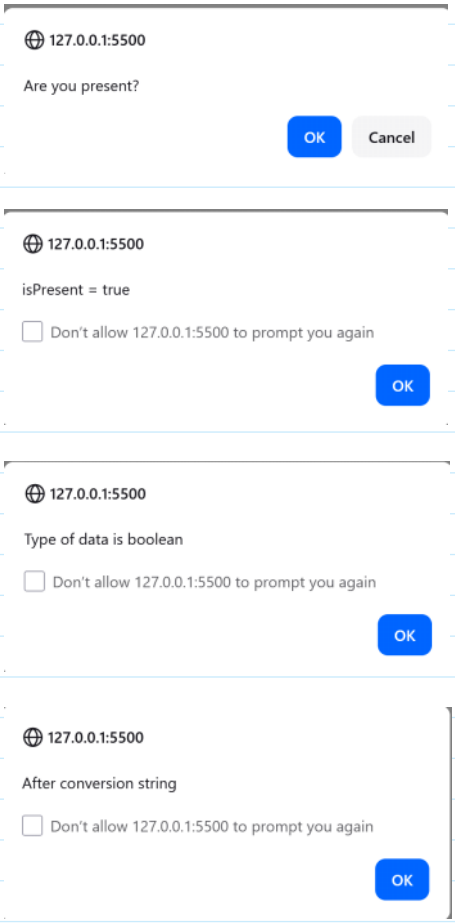
There are also cases when we need to explicitly convert a value to the expected type.

String Conversion

String conversion happens when we need the string from a value.
For example, alert(value) does it to show the value.

```
let isPresent = confirm('Are you present?');
alert(`isPresent = ${isPresent}`);
alert(`Type of data is ${typeof isPresent}`);
let value = String(isPresent);
alert(`After conversion ${typeof value}`);
```

Output:



Numeric Conversion

Numeric conversion in mathematical functions and expression happens automatically. For example when division "/" is applied to non-numbers.

```
console.log("6" / "2");
let str = "123";
console.log(typeof str);
let num = Number(str);
console.log(typeof num);
```

Output:

```
3
string
number
```

Explicit conversion is usually required when we read a value from a string based source like A text form but expected a number to be entered.

If a string is not a valid number, the result of such conversion is Nan. For instance:

```
let age = Number("Twenty")
console.log(age)
```

Output:

```
NaN
```

Numeric Conversion Rule:

Value	Becomes…
undefined	NaN
null	0
true and false	1 and 0
string	White spaces (includes spaces tabs \t, new lines \n, Etc.) from the start and end are removed. If the remaining string is empty, the result is zero, otherwise the number is read from the string. An error gives NaN.

```
console.log(Number("       123        ")); //123
console.log(Number("12345z")); //NaN
console.log(Number(true)); //1
console.log(Number(false)); //0
```

Output:

```
123
NaN
1
0
```

Please note that null and undefined behave differently here: null becomes zero and while undefined becomes NaN.

Most mathematical operators also perform such conversion.

Boolean Conversion
Boolean conversion is the simplest one.
It happens in logical operations. But can also be performed explicitly with a call to Boolean(value).

The conversion rule:
- Values that are intuitively empty like zero and empty string, null, undefined, and NaN becomes false.
- Other values become true.

```
console.log(Boolean(1)); //true
console.log(Boolean(0)); //false
console.log(Boolean("hello")); //true
console.log(Boolean("")) //false
console.log(Boolean(NaN)) //false
console.log(Boolean(null)) //false
console.log(Boolean("0")) //true
console.log(Boolean(" ")) //true
```

Output:

```
true
false
true
false
false
false
true
true
```